3.5 Other Agile Frameworks

The history of software engineering is littered with dozens of obsolete process descriptions and methodologies, modeling methods and notations, tools, and technology. Each flared in notoriety and was then eclipsed by something new and (purportedly) better. With the introduction of a wide array of agile process frameworks—each contending for acceptance within the software development community—the agile movement has followed the same historical path.[6]

As we noted in the last section, one of the most widely used of all agile frameworks is Scrum. But many other agile frameworks have been proposed and are in use across the industry. In this section, we present a brief overview of three popular agile methods: Extreme Programming (XP), Kanban, and DevOps.

3.5.1 The XP Framework

In this section we provide a brief overview of *Extreme Programming* (XP), one of the most widely used approaches to agile software development. Kent Beck [Bec04a] wrote the seminal work on XP.

Extreme Programming encompasses a set of rules and practices that occur within the context of four framework activities: planning, design, coding, and testing. Figure 3.3 illustrates the XP process and notes some of the key ideas and tasks that are associated with each framework activity. The key XP activities are summarized in the paragraphs that follow.

Planning. The planning activity (also called the *planning game*) begins with a requirements activity called *listening*. Listening leads to the creation of a set of "stories" (also called *user stories*) that describe required output, features, and functionality for software to be built. Each *user story* (described in Chapter 7) is written by the customer and is placed on an index card. The customer assigns a *value* (i.e., a priority) to the story based on the overall business value of the feature or function.[7] Members of the XP team then assess each story and assign a *cost*—measured in development weeks—to it. It is important to note that new stories can be written at any time.

Customers and developers work together to decide how to group stories into the next release (the next software increment) to be developed by the XP team. Once a basic *commitment* (agreement on stories to be included, delivery date, and other project matters) is made for a release, the XP team orders the stories that will be developed in one of three ways: (1) all stories will be implemented immediately (within a few weeks), (2) the stories with highest value will be moved up in the schedule and implemented first, or (3) the riskiest stories will be moved up in the schedule and implemented first.

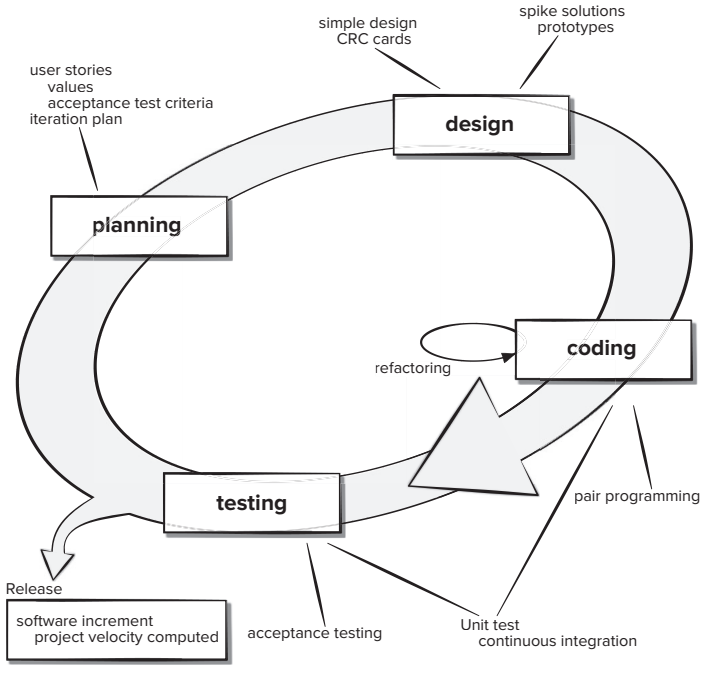
---

6 This is not a bad thing. Before one or more models or methods are accepted as a *de facto* standard, all must contend for the hearts and minds of software engineers. The "winners" evolve into best practice, while the "losers" either disappear or merge with the winning models.

7 The value of a story may also be dependent on the presence of another story.

FIGURE 3.3

The Extreme Programming process



After the first project release (also called a software increment) has been delivered, the XP team computes project velocity. Stated simply, *project velocity* is the number of customer stories implemented during the first release. Project velocity can then be used to help estimate delivery dates and schedule for subsequent releases. The XP team modifies its plans accordingly.

Design. XP design rigorously follows the KIS (keep it simple) principle. The design of extra functionality (because the developer assumes it will be required later) is discouraged.[8]

XP encourages the use of CRC cards (Chapter 8) as an effective mechanism for thinking about the software in an object-oriented context. CRC (class-responsibility-collaborator) cards identify and organize the object-oriented classes[9] that are relevant to the current software increment. CRC cards are the only design work product produced as part of the XP process.

If a difficult design problem is encountered as part of the design of a story, XP recommends the immediate creation of an operational prototype of that portion of the

8 These design guidelines should be followed in every software engineering method, although there are times when sophisticated design notation and terminology may get in the way of simplicity.

9 Object-oriented classes are discussed throughout Part Two of this book.

design. A central notion in XP is that design occurs both before *and after* coding commences. Refactoring—modifying/optimizing the code in a way that does not change the external behavior of the software [Fow00]—means that design occurs continuously as the system is constructed. In fact, the construction activity itself will provide the XP team with guidance on how to improve the design.

Coding. After user stories are developed and preliminary design work is done, the team does *not* move to code, but rather develops a series of unit tests that will exercise each of the stories that is to be included in the current release (software increment).[10] Once the unit test[11] has been created, the developer is better able to focus on what must be implemented to pass the test. Once the code is complete, it can be unit-tested immediately, thereby providing instantaneous feedback to the developers.

A key concept during the coding activity (and one of the most talked-about aspects of XP) is *pair programming.* XP recommends that two people work together at one computer to create code for a story. This provides a mechanism for real-time problem solving (two heads are often better than one) and real-time quality assurance (the code is reviewed as it is created).[12]

As pair programmers complete their work, the code they develop is integrated with the work of others. This "continuous integration" strategy helps uncover compatibility and interfacing errors early.

Testing. The unit tests that are created should be implemented using a framework that enables them to be automated (hence, they can be executed easily and repeatedly). This encourages implementing a regression testing strategy (Chapter 20) whenever code is modified (which is often, given the XP refactoring philosophy). XP *acceptance tests,* also called *customer tests,* are specified by the customer and focus on overall system features and functionality that are visible and reviewable by the customer. They are derived from user stories that have been implemented as part of a software release.

3.5.2 Kanban

The *Kanban* method [And16] is a lean methodology that describes methods for improving any process or workflow. Kanban is focused on change management and service delivery. Change management defines the process through which a requested change is integrated into a software-based system. Service delivery is encouraged by focusing on understanding customer needs and expectations. The team members manage the work and are given the freedom to organize themselves to complete it. Policies evolve as needed to improve outcomes.

---

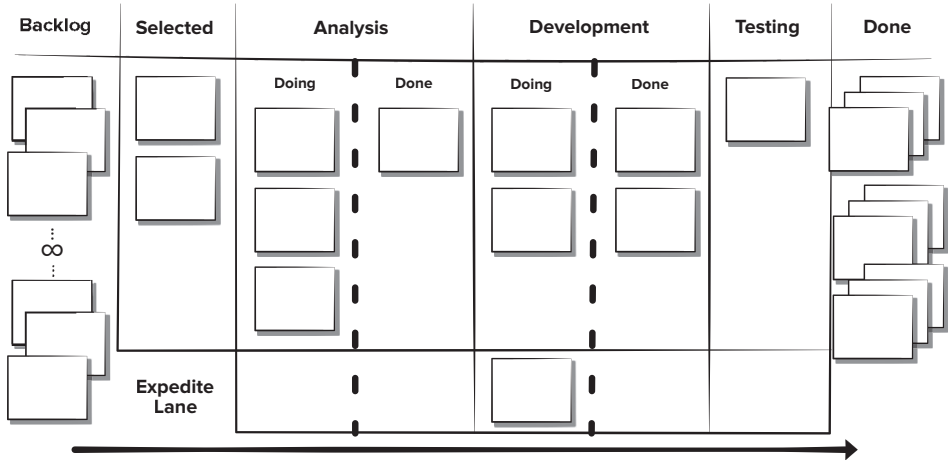
10 This approach is analogous to knowing the exam questions before you begin to study. It makes studying much easier by focusing attention only on the questions that will be asked.

11 Unit testing, discussed in detail in Chapter 20, focuses on an individual software component, exercising the component's interface, data structures, and functionality in an effort to uncover errors that are local to the component.

12 Pair programming has become so widespread throughout the software community that *The Wall Street Journal* [Wal12] ran a front-page story about the subject.

FIGURE 3.4

Kanban board



Kanban originated at Toyota as a set of industrial engineering practices and was adapted to software development by David Anderson [And16]. Kanban itself depends on six core practices:

1. Visualizing workflow using a Kanban board (an example is shown in Figure 3.4). The Kanban board is organized into columns representing the development stage for each element of software functionality. The cards on the board might contain single user stories or recently discovered defects on sticky notes and the team would advance them from "to do," to "doing," and "done" as the project progresses.
2. Limiting the amount of *work in progress* (WIP) at any given time. Developers are encouraged to complete their current task before starting another. This reduces lead time, improves work quality, and increases the team's ability to deliver software functionality frequently to their stakeholders.
3. Managing workflow to reduce waste by understanding the current value flow, analyzing places where it is stalled, defining changes, and then implementing the changes.
4. Making process policies explicit (e.g., write down your reasons for selecting items to work on and the criteria used to define "done").
5. Focusing on continuous improvement by creating feedback loops where changes are introduced based on process data and the effects of the change on the process are measured after the changes are made.[13]
6. Make process changes collaboratively and involve all team members and other stakeholders as needed.

13 The use of process metrics is discussed in Chapter 23.

The team meetings for Kanban are like those in the Scrum framework. If Kanban is being introduced to an existing project, not all items will start in the backlog column. Developers need to place their cards in the team process column by asking themselves: Where are they now? Where did they come from? Where are they going?

The basis of the daily Kanban standup meeting is a task called "walking the board." Leadership of this meeting rotates daily. The team members identify any items missing from the board that are being worked on and add them to the board. The team tries to advance any items they can to "done." The goal is to advance the high business value items first. The team looks at the flow and tries to identify any impediments to completion by looking at workload and risks.

During the weekly retrospective meeting, process measurements are examined. The team considers where process improvements may be needed and proposes changes to be implemented. Kanban can easily be combined with other agile development practices to add a little more process discipline.

3.5.3 DevOps

DevOps was created by Patrick DeBois [Kim16a] to combine Development and Operations. DevOps attempts to apply agile and lean development principles across the entire software supply chain. Figure 3.5 presents an overview of the DevOps workflow. The DevOps approach involves several stages that loop continuously until the desired product exists:

- **Continuous development.** Software deliverables are broken down and developed in multiple sprints with the increments delivered to the quality assurance[14] members of the development team for testing
- **Continuous testing.** Automated testing tools[15] are used to help team members test multiple code increments at the same time to ensure they are free of defects prior to integration.
- **Continuous integration.** The code pieces with new functionality are added to the existing code and to the run-time environment and then checked to ensure there are no errors after deployment.
- **Continuous deployment.** At this stage the integrated code is deployed (installed) to the production environment, which might include multiple sites globally that need to be prepared to receive the new functionality.
- **Continuous monitoring.** Operations staff who are members of the development team help to improve software quality by monitoring its performance in the production environment and proactively looking for possible problems before users find them.

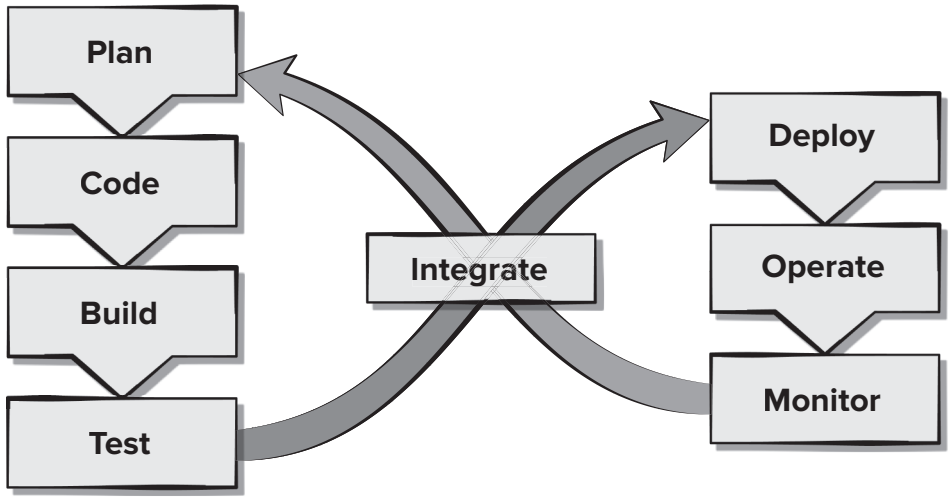
DevOps enhances customers' experiences by reacting quickly to changes in their needs or desires. This can increase brand loyalty and increase market share. Lean approaches like DevOps can provide organizations with increased capacity to innovate

14 The quality assurance is discussed in Chapter 17.

15 Automated testing tools are discussed in Chapter 19.

FIGURE 3.5

DevOps



by reducing rework and allowing shifts to higher business value activities. Products do not make money until consumers have access to them, and DevOps can provide faster deployment time to production platforms [Sha17].

3.6 SUMMARY

In a modern economy, market conditions change rapidly, customer and end-user needs evolve, and new competitive threats emerge without warning. Practitioners must approach software engineering in a manner that allows them to remain agile—to define maneuverable, adaptive, lean processes that can accommodate the needs of modern business.

An agile philosophy for software engineering stresses four key issues: the importance of self-organizing teams that have control over the work they perform, communication and collaboration between team members and between practitioners and their customers, a recognition that change represents an opportunity, and an emphasis on rapid delivery of software that satisfies the customer. Agile process models have been designed to address each of these issues.

Some of the strengths and weaknesses of the agile methods we discussed are summarized in Table 3.1. In previous editions of this book we have discussed many others. The reality is that no agile method is perfect for every project. Agile developers work on self-directed teams and are empowered to create their own process models.

Scrum emphasizes the use of a set of software process patterns that have proven effective for projects with tight time lines, changing requirements, and business criticality. There is no reason why a Scrum team could not adopt the use of a Kanban chart to help organize its daily planning meeting.